

Modulo 1

Buenas prácticas en el desarrollo de modelos

Ingeniería Reproducible para Modelos Eléctricos

El Fin del "modelo_final_v3.ipynb"

El Caos en Operación

- Archivos duplicados: `demanda_norte.py`, `demanda_norte_final.py`, `demanda_ESTE_Sl.ipynb`.
- Miedo a borrar el código que limpiaba bien el Horario de Verano (DST) porque funcionaba ayer.
- Imposible saber qué ingeniero introdujo un sesgo (leakage) al modelo de generación solar.

Con Git (Ingeniería)

- **Un solo archivo** `demanda_norte.py` con todo su historial.
- Fotografías de código (commits) que documentan: "Se agregó temperatura y humedad al modelo XGBoost".
- Experimentación aislada (ramas) para probar si incluir irradiación solar mejora el modelo sin romper producción.

El Ecosistema de Control



Git (Local)

El motor. Software instalado en tu computadora que rastrea el historial de tus notebooks, scripts de Python y datos de configuración meteorológica.



GitHub (Nube)

La plataforma donde se almacena el historial central. Permite que el equipo de pronóstico térmico y el de renovables colaboren en el mismo repositorio.



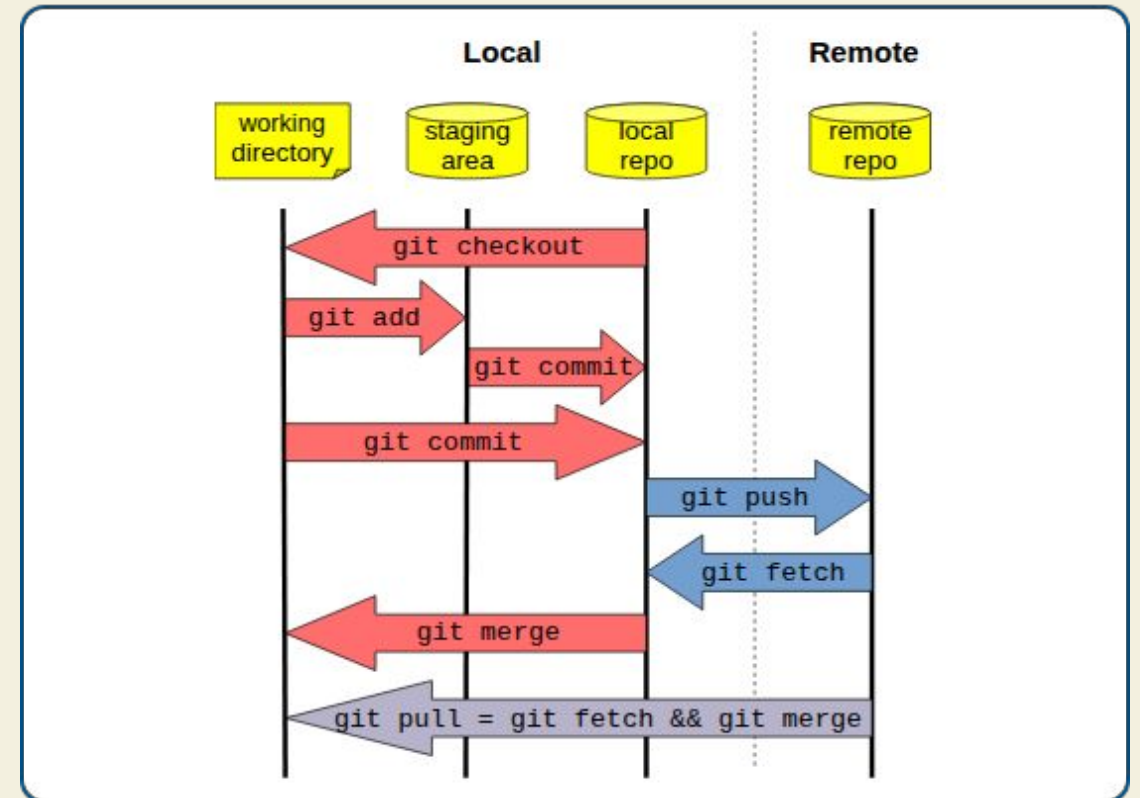
SSH (El Puente)

Protocolo criptográfico que permite a tu terminal local empujar (push) tu código a GitHub de forma segura y sin teclear contraseñas.

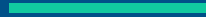
El Flujo de Trabajo Local

Comprender cómo se mueve la información en Git:

- **Working Directory:** Tu VS Code. Aquí rompes y arreglas el script limpieza_clima.py.
- **Staging Area:** Zona de preparación (git add). Solo subes los archivos listos. Quizás preparas limpieza_clima.py pero dejas fuera tus experimentos sueltos.
- **Local Repository:** El historial (git commit). Se guarda la "fotografía" permanente en tu base de datos local.



Autenticación con SSH



Conectando tu terminal a la nube de forma segura

Paso 1: Generar la Llave SSH

Abre tu terminal y genera tus credenciales. *Nota: Si nombraste a tu llave como "ghkey" en lugar del nombre por defecto, úsalo en el comando `ssh-add`.*

```
# Genera un par de llaves criptográficas
$ ssh-keygen -t ed25519 -C "correo@empresa.com"

# Inicia el agente en segundo plano
$ eval "$(ssh-agent -s)"

# Agrega tu llave (ej. ghkey)
$ ssh-add ~/.ssh/ghkey
```



Paso 2: Vincular con GitHub



Copia tu llave pública: Ejecuta en terminal `cat ~/.ssh/ghkey.pub`. Copia todo el texto resultante.



Configura en GitHub: Ve a *Settings* > *SSH and GPG keys*, y haz clic en *New SSH key*.



Pega y Guarda: Pega el texto en el campo 'Key'.



Verifica la conexión: Ejecuta `ssh -T git@github.com`. Si responde "Hi username! You've successfully authenticated...", **¡estás listo!** (Es normal que avise que "no provee acceso shell").

Creación de Repositorios

Estructurando el proyecto de pronóstico

Estrategias de Inicialización

Vía 1: Git Init (De Local a Nube)

Usado cuando ya tienes tu pipeline de datos eléctricos en tu computadora y quieres subirlo a GitHub.

- Requiere inicializar con `git init`.
- Debes crear manualmente un archivo `.gitignore` para evitar subir archivos pesados (.csv de clima).
- Debes enlazar manualmente el repositorio remoto.

Vía 2: Git Clone (De Nube a Local)



El camino más recomendado. Creas el proyecto en GitHub primero y lo descargas pre-configurado. GitHub genera automáticamente el

README.md.

- Puedes decirle a GitHub que añada un `.gitignore` específico para Python.
- El enlace remoto (origin) ya viene configurado.

El Camino Recomendado (Clonar)



Crear en Nube: En GitHub haz clic en "New Repository". Nómbralo (ej. *pronostico-demanda-norte*).



Opciones Iniciales: Marca *"Add a README file"*. Muy importante: en *"Add .gitignore"*, selecciona **Python** para no subir archivos residuales de Colab o de datos pesados por error.



Copiar URL SSH: En tu nuevo repo, ve al botón verde "Code", pestaña **SSH**, y copia el enlace (ej. `git@github.com:user/repo.git`).



Clonar en Local: Abre tu terminal y ejecuta: `git clone git@github.com:...` ¡Ya tienes un sistema bajo control!

El Ciclo Diario del Ingeniero

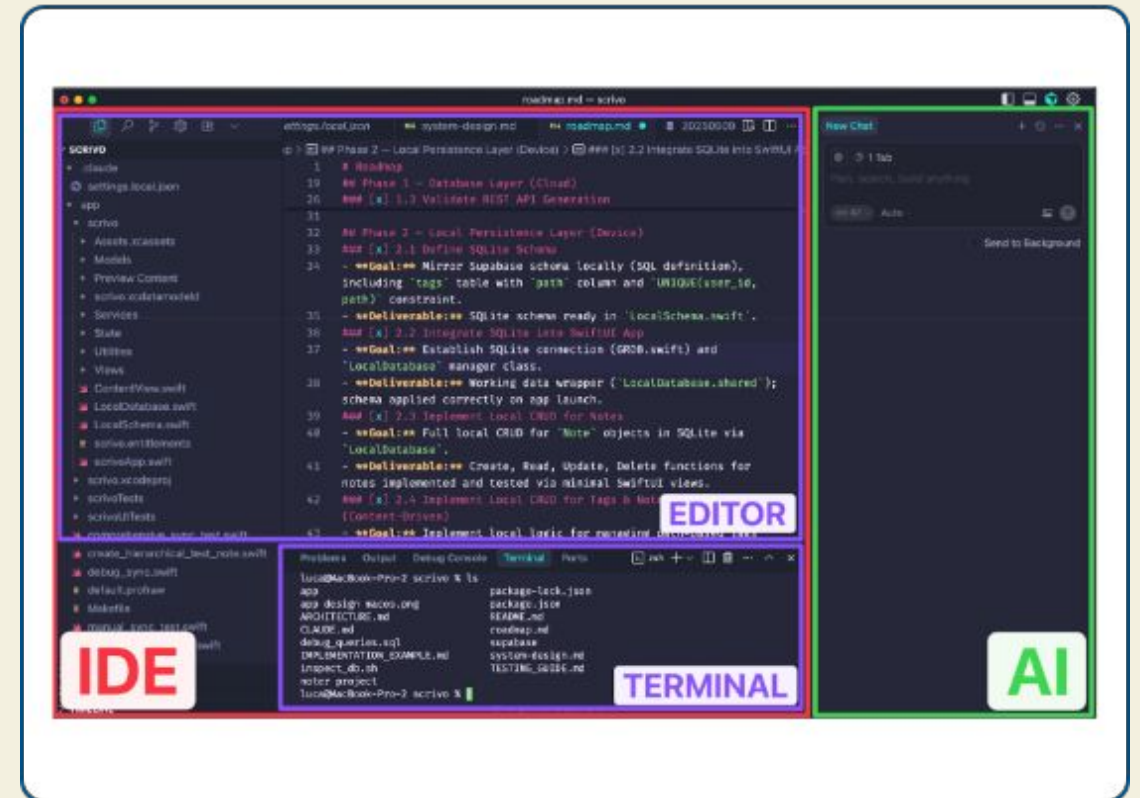
Comandos para guardar tu progreso en tu modelo o script de datos, contextualizado a un día de trabajo real:

```
# 1. ACTUALIZA (Trae cambios de tus colegas)
$ git pull

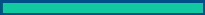
# 2. PREPARA tus correcciones de datos
$ git add limpieza_clima.py

# 3. FOTOGRAFÍA (Commit explicativo)
$ git commit -m "Fix: resuelve interpolación en datos
de radiación solar"

# 4. SUBE a la nube para el equipo
$ git push
```



Ramas (Branches)



Laboratorios de experimentación seguros

¿Por qué usar Ramas?



Líneas de Tiempo

Protegiendo la Operación

En el sector eléctrico, la rama main es tu código en operación (el que predice la carga diaria del CENACE).

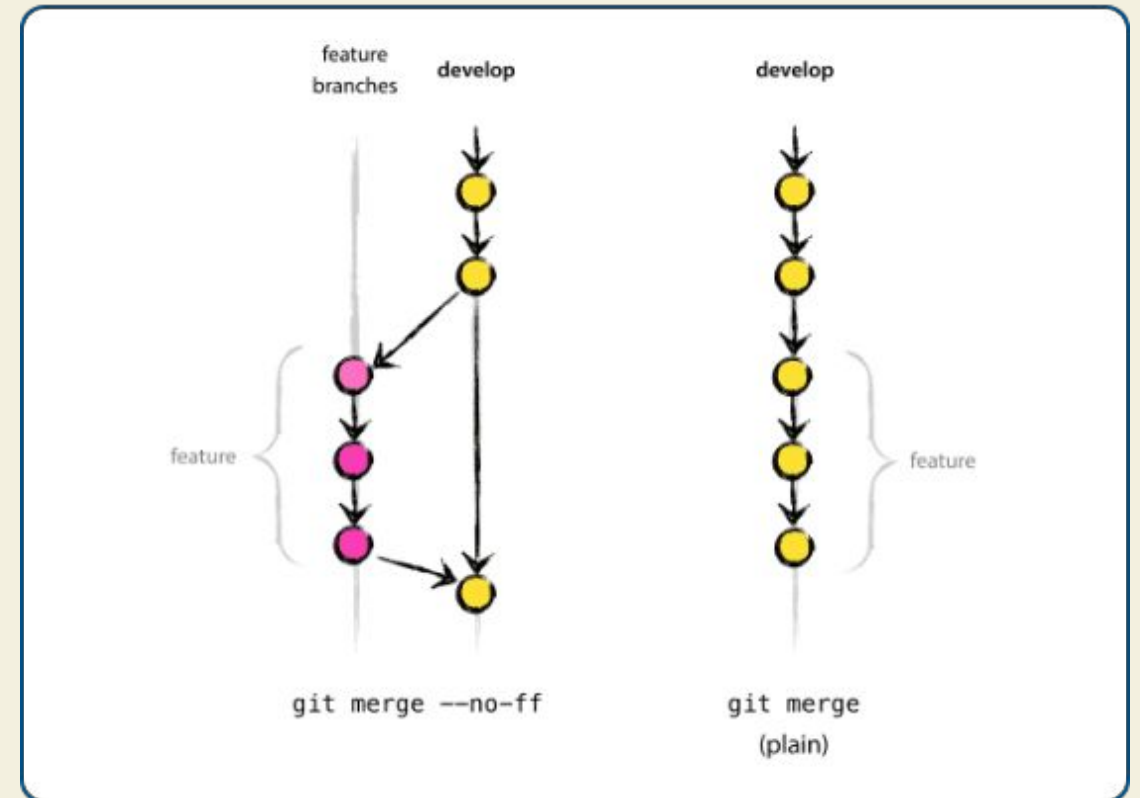
Si quieres probar meter la variable exógena de "humedad relativa" a un nuevo modelo MLForecast, **no lo haces en main.**

Creas una rama paralela. Si el modelo sobreajusta temporalmente o lanza errores físicos, tu código operativo ``main`` permanece 100% intacto y funcional.

El Ciclo de las Ramas

El ciclo de vida para probar nuevas features:

- **Branching:** Creas un clon a partir de un punto estable (ej. rama test-sarima).
- **Committing:** Trabajas de forma aislada, guardando tus métricas de backtesting sin afectar a nadie más.
- **Merging:** Cuando el SARIMA supera el baseline estadístico, "fusionas" (merge) esos avances de vuelta a la línea principal de producción.



Operando con Ramas



Crear & Cambiar

`git branch feat-clima`
(Crea la rama)

`git switch feat-clima`
(Te mueve a ella)



Listar Ramas

`git branch`

Te muestra todas tus ramas. Un asterisco (*) indica en cuál contexto (laboratorio) te encuentras.



Fusionar (Merge)

Muévete al destino:
`git switch main`

Trae los cambios exitosos:
`git merge feat-clima`

Escenario Real: Colaboración

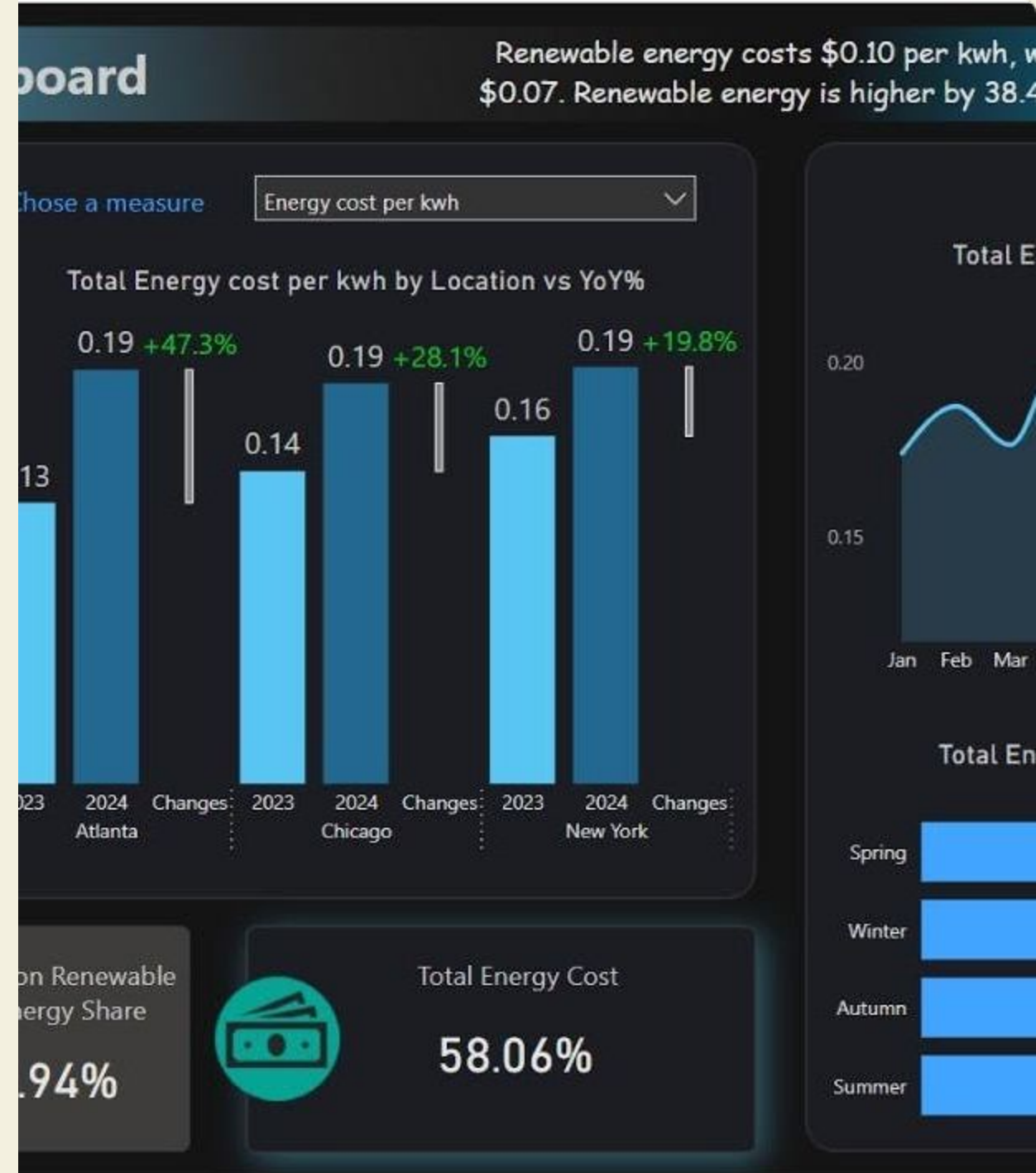
Trabajo en Paralelo

Imagina un proyecto de despacho económico:

Tú trabajas en la rama data-prep diseñando el pipeline de limpieza de datos meteorológicos (resampling, outliers).

Tu colega trabaja en la rama modelo-solar desarrollando el modelo base con StatsForecast.

Ambos trabajan el mismo día sin sobrescribir el código del otro. Al finalizar, Git se encarga de integrar matemáticamente ambos módulos en `main`.



Fusión y Troubleshooting

Integrando el código y resolviendo problemas técnicos

Anatomía de un Merge

1. Preparar el Terreno (`main`)

La regla de oro: nunca fusiones a ciegas. Siempre debes preparar la rama receptora.

- Mueve tu terminal a la rama principal:
`git switch main`
- Asegúrate de tener la versión más reciente de la nube (por si alguien más subió algo):
`git pull origin main`

2. Traer el Experimento

Ahora, absorbe los avances de tu rama de desarrollo.

- Ejecuta el comando de fusión:
`git merge feat-sarima`
- Si no hay código cruzado, Git hará un "Fast-forward" y combinará todo limpiamente.
- Finalmente, súbelo a la nube:
`git push origin main`

Choque de Líneas (Conflictos)

¿Qué pasa cuando la matemática choca?

- Un conflicto ocurre cuando tú y tu colega modificaron **la misma línea del mismo archivo** (ej. la fórmula de limpieza del DST).
- Git es inteligente, pero no adivinará cuál lógica de ingeniería es la correcta para tu pronóstico.
- Git detendrá el `merge` y modificará tu archivo insertando marcadores visuales extraños como <<<<<<<< HEAD.
- El código entra en un estado de "Pausa" hasta que un humano decida qué versión debe prevalecer.



Pasos para Resolver Conflictos

-  **1. Abrir VS Code:** Tu editor detectará los marcadores de conflicto y te dará opciones en pantalla: "Accept Current Change" (tu código de main), "Accept Incoming" (el de la rama) o "Accept Both".
-  **2. Elegir y Limpiar:** Acepta la versión correcta de la fórmula eléctrica o combina ambas lógicas manualmente. Borra manualmente cualquier residuo de símbolos <<<<<< y =====.
-  **3. Marcar como Resuelto:** Una vez que el archivo funciona y el sanity check pasa, notifica a Git que terminaste: `git add limpieza_datos.py`.
-  **4. Finalizar el Merge:** Ejecuta `git commit -m "fix: resuelve conflicto de variables exógenas en limpieza"` para cerrar el ciclo de fusión.

Primeros Auxilios Técnicos



Reset Soft (Deshacer Commit)

`git reset --soft HEAD~1`
Deshace tu última "fotografía" pero **conserva todos los cambios en tus archivos**. Ideal si olvidaste agregar un script de Python al commit o te equivocaste en el mensaje.



Restore (Descartar Cambios)

`git restore modelo.py`
Si arruinaste el código y quieres volver al último estado guardado. **¡Peligro!** Borrará permanentemente lo que no hayas guardado en un commit previo.



Stash (Pausar Trabajo)

`git stash`
Guarda tu trabajo a medias en un "cajón" temporal sin hacer commit. Te permite cambiar de rama, arreglar un bug rápido en main, y usar `git stash pop` para recuperar tu avance.

Convenciones Profesionales de Commits

Un commit no es solo guardar, es explicar a tu "yo" del futuro (y a tu equipo) **qué y por qué** cambió.



fix: Corrige un error en el código.

Ej: fix: soluciona el cruce de timezone en el resampling horario



feat: Añade una nueva característica o modelo.

Ej: feat: incorpora variables de humedad e irradiancia al pipeline



docs: Cambios en la documentación o docstrings.

Ej: docs: agrega descripción de columnas exógenas en el README



test: Añade o corrige pruebas (guardrails).

Ej: test: añade sanity checks para valores negativos de generación

Gestión de Problemas: GitHub Issues

La forma estructurada de reportar y solucionar errores en equipo

¿Qué es un Issue?

El Viejo Paradigma (El Caos)

Enviar un mensaje de WhatsApp a las 11 PM diciendo *"no me corre el modelo"* o *"la tabla da error"*.

- No hay contexto del error ni datos provistos.
- No se incluye el código que falló.
- Es imposible rastrear si el problema se resolvió meses después.

El Paradigma GitHub (Profesional)

Un **Issue** es un "ticket" público en tu repositorio para registrar fallos, proponer mejoras o planear tareas.

- Habilita una discusión técnica ordenada.
- Se puede etiquetar (ej. *bug*, *enhancement*).
- Queda como bitácora histórica para el equipo (ej. "falla de datos del CENACE de 2023").

El "Contrato" para Abrir un Issue

Llegar a la fase práctica diciendo *"no me corre"* no es aceptable. Un Issue perfecto requiere 4 elementos:



Objetivo: ¿Qué intentabas lograr o modelar?

(Ej. *"Trataba de aplicar un remuestreo horario al pronóstico de carga."*)



Expectativa: ¿Cuál era el resultado o la métrica esperada?

(Ej. *"Esperaba un dataframe con 24 filas por día sin valores nulos."*)



Realidad: ¿Qué sucedió exactamente?

(Es indispensable pegar el mensaje de error completo del traceback o adjuntar la gráfica anómala).



Ejemplo Mínimo Reproducible (MRE): Provee un fragmento de código aislado y pequeñísimo junto con datos sintéticos para poder replicar el error de inmediato.

El Ciclo de Vida de un Issue



1. Reporte

En la pestaña "Issues" de GitHub, haces clic en "New Issue". Detallas el problema con tu MRE. GitHub le asignará un número identificador (ej. **#42**).



2. Trabajo (Rama)

Te asignas el Issue. Creas una rama específica en tu terminal:
`git switch -c fix-issue-42`
Haces las correcciones necesarias en el modelo.



3. Resolución

Al hacer tu commit, usas la palabra clave "Fixes":
`git commit -m "fix: corrige división por cero. Fixes #42"`
Al subirlo, GitHub cerrará el ticket automáticamente.

Tarea 1: Aplicando lo Aprendido

De Programación Procedimental a Nodos POO

Tarea 01

Descarga el repo oficial de la clase (solo para obtener el archivo), copia el notebook a **tu propio repositorio personal** y resuelve aplicando POO en una nueva rama.

```
# 1. Clona el repo de la clase (solo para descargar los
archivos)
$ git clone git@github.com:Alveuz/ITSON_AF.git

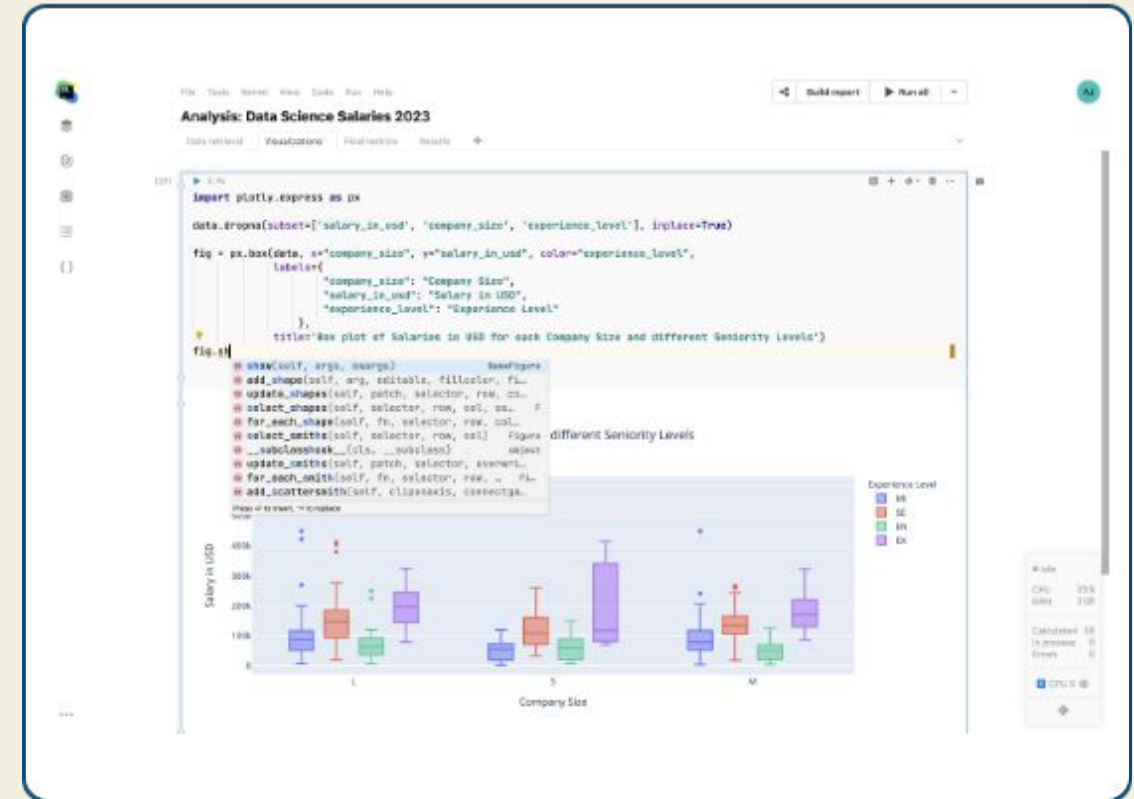
# 2. Copia el notebook a TU propio repositorio y ve a él
# (Asumiendo que nombraste a tu repo 'mi_repo_pronosticos')

$ cp ITSON_AF/Notebook/Clase01_Python...ipynb
../mi_repo_pronosticos/
$ cd ../mi_repo_pronosticos/

# 3. Crea y cámbiate a la rama específica para tu tarea
$ git switch -c feat-tarea01

# 4. Resuelve el Notebook en tu editor y guarda los cambios
$ git add .
$ git commit -m "feat: completa tarea 01 de subestaciones"

# 5. Sube TU rama a TU repositorio en GitHub
$ git push -u origin feat-tarea01
```



¿Dudas Técnicas?

Recuerden la regla de oro: documenten sus problemas estructurados abriendo un *Issue* en el repositorio de GitHub aportando el MRE (Ejemplo Mínimo Reproducible).